Neural Models Lab Report: XOR, Activations, Symmetry, Softmax

Course: CS F407 Artificial Intelligence. **Experiments:** full-batch gradient descent, lr = 0.5, 5000 steps, float64, mean loss over the four examples. All numbers below were measured (seeded initialisation, seed 2 unless stated).

Task 1. Problem specification

Input space X = $\{0,1\}^2$; output space Y = {0,1}. Labelled examples: (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0 (warning = sensors disagree = XOR). In the (x1,x2) plane the class-1 points (0,1),(1,0) sit on one diagonal and the class-0 points (0,0),(1,1) on the other. A single straight line w1*x1 + w2*x2 + b = 0 would have to put both class-1 points on one side and both class-0 points on the other; but the midpoint of the class-1 points and the midpoint of the class-0 points are the same point (0.5,0.5), so any half-plane containing both class-1 points also contains that point and hence cannot exclude both class-0 points. XOR is not linearly separable.

Prediction for a single affine layer + sigmoid: it cannot fit XOR; the best it can do is predict about 0.5 for every input (loss near ln 2 = 0.693). **Measured:** probabilities [0.5, 0.5, 0.5, 0.5], loss 0.6931. Prediction confirmed. *Think About It:* XOR tests the scientific claim that the *kind* of representation (a nonlinear hidden feature space) matters, not the number of parameters.

Task 2. Model design and validation criteria

Architecture: 2 inputs -> 2 hidden units (tanh; sigmoid and ReLU compared later) -> 1 output logit, sigmoid probability, binary cross-entropy (implemented as BCEWithLogitsLoss), gradient descent.

Why a hidden nonlinearity: stacked affine maps collapse to one affine map, which is linear and cannot represent XOR; the nonlinearity lets the hidden layer re-map the four points into a space where they are linearly separable. **Why sigmoid + BCE:** the target is one yes/no answer, sigmoid outputs a probability in (0,1), BCE is the negative log-likelihood of a Bernoulli model, and the logit gradient is simply p - y (no saturation-induced vanishing at the output). **Validation criteria:** (1) final loss much smaller than 0.693; (2) all four thresholded predictions match the labels; (3) first-layer gradient is nonzero and agrees with a finite-difference estimate; (4) repeated runs from several seeds behave consistently.

Think About It: hidden units have no targets. Backpropagation sends the output error back through the output weights (chain rule), so each hidden unit's update is determined by how its output changes the final loss; the useful features emerge from that signal.

Task 3. LLM prompt and corrections

Prompt (replace with the exact prompt you actually used):

```
Generate minimal PyTorch code for a 2-2-1 network on the XOR dataset
(0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0. Hidden activation: tanh. Output: one
logit with BCEWithLogitsLoss. Random weight initialisation, full-batch
gradient descent for a few thousand steps on CPU. Do not change the
architecture or task. After training, print the final loss, all four
probabilities, thresholded labels, and the gradient of the first-layer
weights after backward(). Set a random seed and explain each test in one
sentence.
```

Corrections/notes (edit to match yours): (a) switched to float64 and NumPy-seeded initialisation so results are reproducible and match the finite-difference check; (b) added a finite-difference gradient check and a multi-seed robustness loop, because a single run can hide failures. **Code inspection:** forward pass = `forward()`; scalar loss = `binary_cross_entropy_with_logits`; reverse-mode AD = `loss.backward()`; parameter update = `opt.step()`. Architecture and loss pairing can be verified from the code alone; whether learning actually happens (loss, predictions, gradient correctness, seed

sensitivity) requires running and measuring.

Task 4. Results

A. Basic learning (tanh, seed 2). Initial loss 0.9318, final loss 0.00125. Probabilities [0.0009, 0.9983, 0.9984, 0.0008] -> labels [0,1,1,0]: all four correct.

B. Backprop check. `W1.grad` holds dL/dW1, the derivative of the scalar mean loss with respect to each first-layer weight, obtained by applying the chain rule backwards through output and hidden layers. Autograd-equivalent analytic gradients matched central finite differences to a maximum absolute difference of 3.3e-11 (seed 0, tanh). With a mean loss, the gradient is the average of the four per-example gradients because differentiation is linear: d/dW (1/4 sum_i L_i) = 1/4 sum_i dL_i/dW.

C. Symmetry. With all weights zero, both rows of W1 stay [0,0] for all 5000 steps and the loss stays at 0.6931 (predictions all 0.5). Here the network is stuck completely, because W2 = 0 also blocks the gradient reaching W1 (tanh(0)=0 blocks the gradient to W2). To isolate the symmetry effect I also used identical non-zero weights (all 0.5): the two rows of W1 stayed identical to machine precision at every logged step (1, 10, 100, 1000, 5000), for example [1.428, 1.428] both at step 100, and the loss plateaued near 0.478 with predictions [0.001, 0.666, 0.666, 0.667]: the network cannot solve XOR. Reason: identical hidden units compute the same output, so they receive the same gradient, so gradient descent keeps them identical; two units behave as one.

D. Activation experiment (same initial weights, seed 2):

Hidden activation	Final loss	4/4 correct?	Early ||grad W1|| (step 0)	Successes / 10 seeds
Sigmoid	0.00441	Yes	0.0691	7 / 10
Tanh	0.00125	Yes	0.3218	7 / 10
ReLU	0.47743	No	0.1871	1 / 10

Interpretation. In this experiment, from the same start, the tanh gradient at step 0 was about 4.7 times the sigmoid gradient (0.32 vs 0.069), and tanh reached a lower loss in the same number of steps. Scientifically, sigmoid's derivative is at most 0.25 and it is not zero-centred, so the backward signal is scaled down at each layer, while tanh's derivative reaches 1 at zero. ReLU failed in most seeds here: with only 2 hidden units, a unit whose pre-activation is negative for all four inputs outputs 0 and has derivative 0 ("dead"). In seeds 4 and 8 the initial gradient was exactly 0. The observation is specific to this tiny task (4 points, 2 units, 10 seeds); it does not show ReLU is worse in general. Failed runs of sigmoid and tanh (loss ~0.347, 3/4 correct) were local plateaus from unlucky initialisations, not bugs.

Think About It (distinguishing saturation from dead ReLU): print the pre-activations. A saturated sigmoid/tanh unit has large-magnitude pre-activations (output near 0/1 or +-1) but nonzero, tiny derivative; a dead ReLU has pre-activations that are all negative (output exactly 0, derivative exactly 0).

Task 5. Three-class extension

Output layer changed to 3 logits with softmax + multiclass cross-entropy (hidden layer still 2 tanh units). **Predictions made before running:** final weight matrix W2 has shape (3,2); 3 logits per example; softmax probabilities sum to one because each is exp(z_k) divided by the sum of all exp(z_j); the logit gradient is p - y because d/dz_k of [-z_y + log sum_j exp(z_j)] = p_k - 1[k=y].

Results (seed 2): W2 shape (3,2), logits (4,3), initial loss 1.070, final loss 0.00050, predicted classes [0,1,1,2] (all correct).

Input	P(class 0)	P(class 1)	P(class 2)
(0,0)	0.9994	0.0006	0.0000
(0,1)	0.0002	0.9996	0.0002

(1,0)	0.0002	0.9996	0.0002
(1,1)	0.0000	0.0006	0.9994

For input (0,1): logits [-2.972, 5.475, -2.997] give softmax [0.0002, 0.9996, 0.0002], sum = 1.0. The numerical dL/dz matched (p - y)/N to 2.6e-11. **Shift diagnostic:** adding 100 to all logits changed the (stable) softmax by less than 1e-17. A naive exp/sum implementation returned NaN for a shift of 1000 (exp overflows) while the max-subtracted version was unchanged; stable implementations subtract the maximum logit because softmax is shift-invariant and this keeps exponents at most 0.

Think About It: with tens of thousands of classes, softmax, cross-entropy and the p - y gradient stay the same. What changes is the surrounding architecture (deep transformer layers, huge embedding and output matrices, context, and the cost of computing the full softmax).

Reflection questions

1. Adding affine-only depth does not help XOR; nonlinearity between layers is what gives the extra representational power. Depth without nonlinearity is still a single linear map.

2. The loss fell from 0.93 to 0.00125 and the four predictions flipped to the correct labels, and the analytic gradient matched finite differences (3.3e-11). So the gradient was correct and pointed downhill, rather than merely nonzero (the symmetric-init run had nonzero gradients yet plateaued at 0.478).

3. Identical/zero initial weights make the hidden units compute identical outputs and receive identical gradients, so they never differentiate (and with zero W2, W1 gets no gradient at all). Random initialisation breaks the symmetry.

4. Engineering observation: tanh gave a larger early gradient (0.32) than sigmoid (0.069) from the same start; ReLU gave a zero gradient in some seeds. Scientific explanation: sigmoid derivatives are at most 0.25 and shrink the backward signal, tanh is zero-centred with derivative up to 1, and inactive ReLU units have derivative exactly 0.

5. The loss must be a proper likelihood for the output distribution (Bernoulli for sigmoid, categorical for softmax), which also gives the clean gradient p - y; mismatched pairs distort the gradient or give meaningless probabilities.

6. Productivity: the LLM produced the boilerplate training loop and helped choose BCEWithLogitsLoss. Verification essential: checking that the gradient was correct (finite differences), running multiple seeds, and noticing that a single 'successful' run hid failures and that the zero-init test needed an extra non-zero symmetric control. (Edit this to reflect your own experience.)

7. Keep: loss monitoring, prediction checks, multi-seed runs, gradient-norm and activation-statistics monitoring, and a finite-difference check on a few parameters. Too expensive at scale: exhaustive finite-difference checks over all parameters (cost grows with the parameter count) and exhaustive multi-seed retraining.